PROJECT REPORT

On

BATTERY MANAGEMENT SYSTEM(BMS) SIMULATION

NAME : Ganesh Sambhaji Kate.

CODE :

```
"""
Battery Management System (BMS) Simulation
Project using Python

Features:
1. SOC estimation
2. SOH estimation
3. Battery voltage monitoring
4. Temperature modelling
5. Warning alerts
6. Graph plotting
7. CSV data export

Educational simulation only.
Not suitable for controlling a real battery.
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Battery parameters
NOMINAL_CAPACITY_AH = 50.0
INITIAL_SOC = 65.0
INITIAL_SOH = 92.0
INITIAL_TEMPERATURE = 25.0
AMBIENT_TEMPERATURE = 25.0
```

```
MIN_VOLTAGE = 10.5
MAX_VOLTAGE = 14.6
MAX_TEMPERATURE = 45.0

INTERNAL_RESISTANCE = 0.035
THERMAL_RESISTANCE = 0.12
THERMAL_TIME_CONSTANT = 900.0

def clamp(value, minimum, maximum):
  """Limit a value within a given range."""
  return max(minimum, min(value, maximum))

def estimate_voltage(soc, current):
  """Estimate voltage using a simplified model."""

  soc_fraction = clamp(soc / 100.0, 0.0, 1.0)

  open_circuit_voltage = (
    MIN_VOLTAGE
    + (MAX_VOLTAGE - MIN_VOLTAGE) * soc_fraction
  )

  # Positive current means discharge.
  voltage = (
    open_circuit_voltage
    - current * INTERNAL_RESISTANCE
  )
```

```
    return clamp(voltage, 0.0, MAX_VOLTAGE + 0.5)

def simulate_bms():
    """Simulate battery charging and discharging."""

    duration_minutes = 240
    step_seconds = 10

    total_seconds = duration_minutes * 60

    time_values = np.arange(
        0,
        total_seconds + step_seconds,
        step_seconds
    )

    dt_hours = step_seconds / 3600.0

    # Example current profile:
    # First 60 minutes: discharge at 8 A
    # Next 60 minutes: charge at 6 A
    # Last 120 minutes: discharge at 4 A

    current_values = np.where(
        time_values < 3600,
        8.0,
        np.where(time_values < 7200, -6.0, 4.0)
```

```
)

available_capacity = (
    NOMINAL_CAPACITY_AH * INITIAL_SOH / 100.0
)

soc = INITIAL_SOC
temperature = INITIAL_TEMPERATURE

records = []

for time_sec, current in zip(
    time_values, current_values
):

    # Calculate battery voltage
    voltage = estimate_voltage(soc, current)

    # Calculate heat generated inside the battery
    heat_power = current ** 2 * INTERNAL_RESISTANCE

    temperature_rise = (
        heat_power * THERMAL_RESISTANCE
    )

    # Simplified thermal model
    temperature += (
        step_seconds / THERMAL_TIME_CONSTANT
```

```
    ) * (
        AMBIENT_TEMPERATURE
        + temperature_rise
        - temperature
    )

    # Store current readings
    records.append({
        "Time_min": time_sec / 60.0,
        "Current_A": current,
        "Voltage_V": voltage,
        "SOC_percent": soc,
        "SOH_percent": INITIAL_SOH,
        "Temperature_C": temperature,
        "Capacity_Ah": available_capacity
    })

    # SOC calculation using coulomb counting
    # Positive current = discharge
    soc_change = (
        current * dt_hours / available_capacity
    ) * 100.0

    soc = clamp(soc - soc_change, 0.0, 100.0)

return pd.DataFrame(records)
```

```
def display_results(df):
    """Display final battery parameters."""

    print("\n====================================")
    print(" BATTERY MANAGEMENT SYSTEM")
    print("====================================")

    print(
        f"Simulation duration: "
        f"{df['Time_min'].iloc[-1]:.1f} minutes"
    )

    print(
        f"Initial SOC: "
        f"{df['SOC_percent'].iloc[0]:.2f}%"
    )

    print(
        f"Final SOC: "
        f"{df['SOC_percent'].iloc[-1]:.2f}%"
    )

    print(
        f"Estimated SOH: "
        f"{df['SOH_percent'].iloc[-1]:.2f}%"
    )

    print(
```

```
        f"Final Voltage: "
        f"{df['Voltage_V'].iloc[-1]:.2f} V"
    )

    print(
        f"Final Temperature: "
        f"{df['Temperature_C'].iloc[-1]:.2f} °C"
    )

    print(
        f"Available Capacity: "
        f"{df['Capacity_Ah'].iloc[-1]:.2f} Ah"
    )


def check_alerts(df):
    """Check voltage and temperature thresholds."""

    print("\n========== BMS ALERTS ==========")

    low_voltage = (
        df["Voltage_V"] < MIN_VOLTAGE
    ).any()

    high_voltage = (
        df["Voltage_V"] > MAX_VOLTAGE
    ).any()
```

```
    high_temperature = (
        df["Temperature_C"] > MAX_TEMPERATURE
    ).any()

    if low_voltage:
        print("WARNING: Low battery voltage detected.")

    if high_voltage:
        print("WARNING: High battery voltage detected.")

    if high_temperature:
        print("WARNING: High battery temperature detected.")

    if not (
        low_voltage
        or high_voltage
        or high_temperature
    ):
        print("No configured threshold alerts detected.")

    print("Note: This is an educational simulation.")



def plot_graphs(df):
    """Plot battery monitoring graphs."""

    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
```

```
fig.suptitle(
    "Battery Management System Simulation",
    fontsize=15
)

# Graph 1: State of Charge
axes[0, 0].plot(
    df["Time_min"],
    df["SOC_percent"]
)

axes[0, 0].set_title("State of Charge (SOC)")
axes[0, 0].set_xlabel("Time (minutes)")
axes[0, 0].set_ylabel("SOC (%)")
axes[0, 0].set_ylim(0, 100)
axes[0, 0].grid(True)

# Graph 2: Battery voltage
axes[0, 1].plot(
    df["Time_min"],
    df["Voltage_V"]
)

axes[0, 1].axhline(
    MIN_VOLTAGE,
    linestyle="--",
    label="Minimum voltage"
)
```

```
axes[0, 1].axhline(
   MAX_VOLTAGE,
   linestyle="--",
   label="Maximum voltage"
)

axes[0, 1].set_title("Battery Voltage")
axes[0, 1].set_xlabel("Time (minutes)")
axes[0, 1].set_ylabel("Voltage (V)")
axes[0, 1].legend()
axes[0, 1].grid(True)

# Graph 3: Battery temperature
axes[1, 0].plot(
   df["Time_min"],
   df["Temperature_C"]
)

axes[1, 0].axhline(
   MAX_TEMPERATURE,
   linestyle="--",
   label="Maximum temperature"
)

axes[1, 0].set_title("Battery Temperature")
axes[1, 0].set_xlabel("Time (minutes)")
axes[1, 0].set_ylabel("Temperature (°C)")
```

```
    axes[1, 0].legend()
    axes[1, 0].grid(True)

    # Graph 4: State of Health
    axes[1, 1].plot(
        df["Time_min"],
        df["SOH_percent"]
    )

    axes[1, 1].set_title("State of Health (SOH)")
    axes[1, 1].set_xlabel("Time (minutes)")
    axes[1, 1].set_ylabel("SOH (%)")
    axes[1, 1].set_ylim(0, 100)
    axes[1, 1].grid(True)

    plt.tight_layout()
    plt.show()


def main():
    """Main program."""

    try:
        df = simulate_bms()

        display_results(df)

        check_alerts(df)
```

```
        # Save simulation readings to CSV
        df.to_csv(
            "bms_simulation_results.csv",
            index=False
        )

        print(
            "\nData saved to "
            "bms_simulation_results.csv"
        )

        # Display graphs
        plot_graphs(df)

    except Exception as error:
        print("An error occurred:", error)


if __name__ == "__main__":
    main()
```

OUTPUT:

```
====================================
 BATTERY MANAGEMENT SYSTEM
====================================
Simulation duration: 240.0 minutes
Initial SOC: 65.00%
```

Final SOC: 43.26%

Estimated SOH: 92.00%

Final Voltage: 12.13 V

Final Temperature: 25.07 °C

Available Capacity: 46.00 Ah

========== BMS ALERTS ==========

No configured threshold alerts detected.

Note: This is an educational simulation.

Data saved to bms_simulation_results.csv

